

Blueprint de Excelência

Agente de revisão e aprovação automática + portal Docusaurus versionado

Implementação completa para GitHub, GitHub Actions, Python/FastAPI e documentação viva governada por PR, release, políticas de qualidade e versionamento controlado.

Propósito do documento

Entregar um desenho de produção, com guardrails de segurança, política de auto-approval sem relaxar a integridade do repositório, Docusaurus operando como portal profissional de documentação e automação documental executando em todo PR com congelamento de versões em releases.

Status	Pronto para implantação corporativa
Escopo	Revisão automática de PR, quality gates, Docusaurus, exportação OpenAPI, deploy e versionamento
Público-alvo	Engenharia de software, dados, plataforma, DevOps, arquitetura e liderança técnica
Stack-base assumida	GitHub + GitHub Actions + Python 3.12 + Poetry + pytest + Ruff + Docusaurus v3
Data	01/04/2026

Premissa central: o agente jamais recebe permissão para mascarar falhas, contornar proteção de branch, aprovar mudanças de alto risco sem política explícita ou executar código não confiável em workflow privilegiado. A automação acelera; a governança continua no comando. [R7][R8][R12][R13][R15]

Sumário estrutural

Seção	Conteúdo
1	Objetivo, premissas e princípios inegociáveis
2	Arquitetura-alvo do fluxo de PR e documentação
3	Governança do repositório, branch protection e rulesets
4	Passo a passo técnico de implantação do agente
5	Implementação do Docusaurus, geração documental e deploy
6	Versionamento, releases e estratégia de congelamento
7	Operação contínua, failure modes e métricas
Apêndices	Arquivos completos de workflow, scripts e configuração
Referências	Fontes oficiais e componentes de suporte

Como usar este blueprint

Primeiro implemente o fluxo de quality gates e branch protection. Em seguida conecte o agente de auto-approval. Depois implante o Docusaurus, a exportação OpenAPI e o build documental no mesmo PR CI. Por fim, feche o ciclo com deploy e versionamento orientado por release.

1. Objetivo, premissas e princípios inegociáveis

Este blueprint assume que o repositório principal vive no **GitHub**, usa **GitHub Actions** como esteira padrão e precisa obter três capacidades simultâneas: inspeção automática de qualidade, aprovação automática controlada e documentação profissional viva. As decisões abaixo foram escolhidas porque o GitHub permite branch protection, rulesets, required status checks, reviews, CODEOWNERS e aprovações geradas por automação quando isso é habilitado na configuração do repositório. [R7][R8][R9][R10][R11][R15]

No eixo documental, a base recomendada é o **Docusaurus v3**, com suporte de primeira classe a TypeScript, versionamento de docs, build estático, deploy por GitHub Actions e publicação previsível. O portal é tratado como produto de engenharia, não como wiki informal. [R1][R2][R3][R4][R5][R6][R16]

Princípio	Decisão arquitetural
Segurança	Workflow privilegiado nunca faz checkout do código do PR e nunca executa código não confiável.
Integridade	O agente só aprova depois que o workflow de PR CI conclui com sucesso.
Governança	Mudanças de alto risco continuam exigindo intervenção humana via CODEOWNERS e labels de bloqueio.
Rastreabilidade	Toda aprovação automática deixa review formal no PR com política aplicada.
Documentação viva	Todo PR constrói docs, exporta OpenAPI e valida o portal.
Versionamento	Versões são congeladas por release com estratégia explícita; não se cria versão para todo patch sem necessidade.

Decisão de engenharia recomendada

Separe o fluxo em dois domínios: um workflow não privilegiado para build, lint, sintaxe, tipos, testes e docs; e um workflow privilegiado, disparado somente após sucesso do anterior, responsável por aprovar o PR quando a política permitir. Essa divisão reduz superfície de ataque e segue a modelagem recomendada para uso seguro do GITHUB_TOKEN e de eventos privilegiados. [R12][R13][R14]

2. Arquitetura-alvo do fluxo de PR e documentação

A topologia proposta tem seis blocos: autor abre PR; workflow PR CI executa quality gates e build documental; branch protection aguarda status green; workflow privilegiado avalia política; o agente aprova somente quando o conjunto é elegível; ao merge em main o portal Docusaurus é reconstruído e publicado. Required status checks e reviews continuam sendo o mecanismo de merge gate do GitHub. [R7][R9][R10][R12][R16]

Bloco	Responsabilidade	Permissão recomendada
PR CI	Lint, sintaxe, testes, mypy opcional, exportação OpenAPI, build Docusaurus e upload do artefato de docs.	contents: read
Docs guardrails	Falhar PR quando comportamento público mudou e documentação curada não acompanhou.	contents: read
Branch rules	Impedir merge com checks falhos, conversas abertas e branches fora de política.	Configuração do repo
Approval bot	Ler metadados do PR, aplicar política e registrar review APPROVE quando elegível.	pull-requests: write
Deploy docs	Buildar portal em main e publicar no GitHub Pages.	pages: write + id-token: write
Release docs	Congelar versão documental e abrir PR de versionamento em releases.	contents: write + pull-requests: write

A arquitetura abaixo assume um portal em **docs-site/** dentro do mesmo monorepo ou repositório principal. Essa decisão simplifica a origem da verdade: PR, código, contrato OpenAPI, guias operacionais e automações vivem no mesmo ciclo de revisão e publicação. Docusaurus oferece isso com build estático, MDX, plugin docs, versioning CLI e deploy por GitHub Actions. [R1][R4][R6]

PR aberto

- > .github/workflows/pr-ci.yml
 - > quality-gates (ruff, compileall, pytest, mypy opcional)
 - > docs-gates (export_openapi.py, gen-api-docs, docusaurus build)
- > required status checks = green
- > .github/workflows/pr-approval.yml (workflow_run)
 - > approval_policy.py
 - > review APPROVE somente se elegível
- > merge em main
 - > .github/workflows/docs-deploy.yml
 - > build portal
 - > deploy GitHub Pages
- > push de tag/release
 - > .github/workflows/docs-version-pr.yml
 - > docusaurus docs:version <versao>
 - > cria branch + PR de congelamento documental

3. Governança do repositório, branch protection e rulesets

Antes de qualquer automação, configure o repositório. GitHub branch protection e rulesets permitem exigir checks, reviews, impedir force-push, resolver conversas e definir políticas por branches e tags. Sem isso, a automação fica bonita porém estruturalmente frágil. [R7][R8][R9][R10]

1. Ative GitHub Actions no repositório e defina o GITHUB_TOKEN com permissão padrão de leitura para contents.
2. Em Settings > Actions > General, habilite “Allow GitHub Actions to create and approve pull requests” somente porque o approval bot precisa registrar reviews. [R15]
3. Proteja a branch principal com required status checks, conversation resolution e bloqueio de force-push. [R7][R9]
4. Crie CODEOWNERS para áreas críticas: segurança, CI, infraestrutura, banco, docs configuration e contratos públicos. [R11]
5. Modele labels de risco: security, breaking-change, needs-human-review, db-migration, infra-change.
6. Defina explicitamente quais diretórios nunca são elegíveis para auto-approval.
7. Se o repositório recebe PRs de forks, trate-os como fluxo mais restritivo; o approval bot não deve auto-aprovar PR de fork. [R12]

Área	Política recomendada
.github/workflows/	Sempre revisão humana; nunca auto-approve.
infra/, terraform/, helm/, Dockerfile*	Sempre revisão humana.
migrations/, alembic/	Sempre revisão humana e, se aplicável, DBA ou owner da aplicação.
src/ domínio de negócio de baixo risco	Elegível a auto-approval se todos os gates estiverem green e sem labels de bloqueio.
docs-site/docs/ e docs/	Elegível; CI deve validar build e links.
openapi/ e contratos públicos	Human-in-the-loop quando houver breaking changes.

Regra de ouro

Auto-approval não deve ser entendido como substituição universal da revisão humana. Ele é excelente para alterações de baixo risco, PRs rotineiros, fixes triviais e mudanças coerentes com uma política formal. Áreas que alteram segurança, infraestrutura, esquema de dados, contratos públicos ou o próprio pipeline continuam sob revisão humana mandatória.

4. Passo a passo técnico de implantação do agente

A implantação abaixo foi organizada em fases para minimizar acoplamento e facilitar rollback. O repositório primeiro precisa provar que consegue falhar corretamente; só depois ele recebe o poder de aprovar. Essa ordem evita institucionalizar uma automação que aprova PRs sem um gate confiável.

4.1 Fase A — preparar a base do repositório

8. Garanta estrutura mínima: src/, tests/, .github/workflows/, .github/scripts/, docs-site/.
9. Padronize dependências em lockfile e formalize comandos em pyproject.toml e package.json.
10. Crie os labels operacionais e o arquivo CODEOWNERS.
11. Habilite protection/rulesets antes de publicar o approval bot.

4.2 Fase B — workflow PR CI não privilegiado

O PR CI roda em evento **pull_request**. Ele deve fazer checkout do código do PR, instalar dependências, validar lint, sintaxe, testes e a documentação. Como é um workflow não privilegiado, ele não precisa de segredos nem permissões de escrita. Isso é consistente com o modelo de segurança do GitHub para GITHUB_TOKEN e PRs, incluindo forks. [R12][R13][R14]

- Lint: Ruff ou ferramenta equivalente.
- Sintaxe: `python -m compileall` em `src` e `tests`.
- Testes: `pytest` com relatório JUnit.
- Tipos: `mypy` opcional, porém recomendado em código central.
- Docs: exportar OpenAPI, gerar referência, buildar o Docusaurus e subir artefato preview.
- Freshness: falhar se o comportamento público mudou e nenhuma doc curada foi tocada.

4.3 Fase C — workflow privilegiado de aprovação

O workflow privilegiado deve ser disparado por **workflow_run** após o sucesso do PR CI. O GitHub documenta que um workflow iniciado por `workflow_run` pode acessar segredos e write tokens mesmo se o anterior não pôde; exatamente por isso o workflow privilegiado não deve executar código vindo do PR. Ele deve operar apenas com metadados do evento e chamadas à API. [R12][R13]

- Não fazer `actions/checkout` do HEAD do PR nesse workflow.
- Ler o número do PR via `github.event.workflow_run.pull_requests[0].number`.
- Consultar PR, labels, arquivos alterados e reviews pela API REST.
- Aplicar política: rejeitar forks, drafts, labels bloqueantes e caminhos críticos.
- Registrar review `APPROVE` somente quando elegível.
- Encerrar silenciosamente quando o PR não for elegível; a ausência de aprovação é parte da política.

4.4 Fase D — observabilidade e operação

- Deixe mensagem explícita na aprovação automática indicando quais gates foram satisfeitos.
- Versione a política do bot junto do repositório; `policy drift` não pode ficar implícito.
- Acompanhe taxa de auto-approval, taxa de false positive, tempo médio até merge e percentual de PRs bloqueados por docs.

Ponto crítico de segurança

GitHub alerta para o risco de executar código não confiável em `pull_request_target` e `workflow_run`. O desenho proposto elimina esse problema no trecho privilegiado, porque ele não faz `checkout` do PR e não chama scripts do branch do autor. [R12][R13]

5. Implementação do Docusaurus, geração documental e deploy

O portal documental deve ser um site estático versionado junto do produto. Docusaurus foi desenhado justamente para docs, blogs e páginas estáticas em React/MDX, com suporte a TypeScript, versionamento e deploy orientado por CI. [R1][R3][R4][R6]

5.1 Estrutura recomendada

```
repo/
├── .github/
│   └── scripts/
│       └── approval_policy.py
```

```

├── docs_guardrails.py
├── workflows/
│   ├── pr-ci.yml
│   ├── pr-approval.yml
│   ├── docs-deploy.yml
│   └── docs-version-pr.yml
├── src/
├── tests/
├── scripts/
│   └── export_openapi.py
├── docs-site/
│   ├── docs/
│   │   ├── index.md
│   │   ├── architecture/
│   │   ├── standards/
│   │   ├── runbooks/
│   │   └── api/          # gerado automaticamente
│   ├── static/
│   ├── src/
│   ├── docusaurus.config.ts
│   ├── sidebars.ts
│   ├── package.json
│   ├── versions.json    # aparece após o primeiro freeze
│   ├── versioned_docs/
│   └── versioned_sidebars/

```

5.2 Inicialização e TypeScript

A inicialização recomendada usa o template clássico com TypeScript: ``npx create-docusaurus@latest docs-site classic --typescript``. O Docusaurus documenta suporte de primeira classe a TypeScript e pede TypeScript 5.1 ou superior. [R2][R3]

Para um portal corporativo limpo, a recomendação é usar **routeBasePath**: `"/"` e transformar o portal em docs-first. O conteúdo curado entra em ``docs/``; a referência de API entra em ``docs/api/`` gerada automaticamente a partir do OpenAPI.

5.3 OpenAPI e documentação de API

Em stack FastAPI, a melhor origem da verdade para referência de API é o próprio schema OpenAPI exportado pelo app. O portal então converte esse contrato em MDX. Para isso, o componente mais maduro e amplamente usado hoje é o plugin comunitário ``docusaurus-plugin-openapi-docs``, compatível com Docusaurus v3 e capaz de gerar MDX a partir de Swagger 2.0/OpenAPI 3.x. [R20]

- Curadoria humana: visão geral, arquitetura, runbooks, troubleshooting, ADRs e standards.
- Geração automática: endpoints, schemas, exemplos e contratos a partir do OpenAPI.
- Build por PR: exportar schema, regenerar referência e validar o site inteiro.
- Produção: publicar apenas build validado em main.

5.4 Deploy e metadados de atualização

O build em produção usa GitHub Pages com custom workflow. GitHub documenta o uso de ``actions/configure-pages``, ``actions/upload-pages-artifact`` e ``actions/deploy-pages``, além das permissões ``pages: write`` e ``id-token: write`` no job de deploy. [R16]

Para exibir “last updated” corretamente, Docusaurus informa que o build precisa de histórico Git completo; por isso os workflows de docs devem usar checkout com ``fetch-depth: 0``. [R5][R17]

5.5 Search, UX e governança editorial

Docusaurus tem suporte oficial ao Algolia DocSearch para sites públicos de documentação. Para portais internos, trate busca como decisão separada: se o site não puder ser rastreado publicamente, use um mecanismo interno compatível com a política da empresa. [R6]

- Navbar enxuta com Home, Architecture, Standards, Runbooks e API.
- Dropdown de versões no topo quando o portal já estiver congelando releases.
- Edit URL apontando para o repositório para manter docs-as-code.
- showLastUpdateTime e showLastUpdateAuthor ativados para auditoria operacional.
- Padrão editorial com templates por tipo de documento: runbook, ADR, API guide, incident note e onboarding.

6. Versionamento, releases e estratégia de congelamento

Docusaurus possui versioning CLI nativa. Ao rodar ``docs:version <versão>``, ele congela a pasta ``docs/``, cria a sidebar versionada e atualiza ``versions.json``. [R4]

O próprio Docusaurus recomenda versionar só quando necessário, porque versionar tudo aumenta custo de manutenção. O padrão recomendado aqui é: versão documental obrigatória em major e minor que mudem comportamento público; patch só quando a documentação realmente divergir. [R4]

Tipo de release	Congelar docs?	Observação
Major	Sim	Mudança estrutural, contratos, fluxo de uso ou semântica relevante.
Minor	Sim	Novos recursos, novos endpoints, novos comportamentos públicos.
Patch sem mudança pública	Não	Atualize a versão corrente; não duplique sem necessidade.
Patch com mudança pública material	Avaliar	Congele se clientes/usuários precisarem navegar aquela fotografia.

Para preservar branch protection, o freeze da versão documental não deve empurrar commit diretamente para main sem política explícita. O fluxo corporativo mais limpo é: release/tag dispara workflow, workflow gera branch de automação com ``docs:version``, comita ``versioned_docs/``, ``versioned_sidebars/`` e ``versions.json``, depois abre PR para main. Ao merge, o deploy publica a nova fotografia. Isso torna o freeze auditável e alinhado ao processo de change management.

Estratégia “quase perfeita” de engenharia

Se a organização já possui release PR ou release train, encaixe o freeze da documentação dentro desse PR antes da tag final. Essa é a forma mais limpa de manter produto, changelog, release notes, contratos e docs congelados no mesmo ato de governança.

7. Operação contínua, failure modes e métricas

Uma automação excelente não é medida apenas pelo que faz quando tudo está certo, mas pela forma como falha. O desenho abaixo privilegia “fail closed”: quando houver dúvida, risco elevado ou sinal contraditório, o bot não aprova e o PR segue para triagem humana.

Falha	Comportamento correto
PR CI falhou	Sem aprovação automática. O review do bot não ocorre.

Workflow privilegiado sem PR associado	Encerrar com sucesso técnico e sem efeito colateral.
Mudança em caminho protegido	Sem aprovação automática; reviewers humanos permanecem responsáveis.
PR draft	Sem aprovação automática até sair do draft.
Fork / Dependabot / fonte externa	Fluxo mais restritivo; nunca executar aprovação cega.
Docs build falhou	Merge bloqueado; portal não publica artefato inválido.
Freeze release gerou conflito	Abrir PR de automação e deixar resolução humana explícita.

Métrica	Meta inicial
Tempo médio de feedback do PR CI	< 10 min
Taxa de PR elegível ao auto-approval	20% a 50%, dependendo do perfil do repositório
Falhas de docs por PR	Tendência de queda após templates e guardrails
Tempo médio de publicação da documentação após merge	< 5 min
Número de versões publicadas ativas	2 a 3 em produção, salvo necessidade regulatória

Feche o ciclo com um padrão operacional simples: toda mudança pública relevante gera docs; toda docs é validada no PR; toda release relevante congela versão; toda aprovação automática deixa trilha formal; toda área crítica continua coberta por humans-in-the-loop.

Resultado esperado

Quando bem implantado, o repositório passa a se comportar como um sistema de engenharia maduro: PR pequeno e saudável flui sozinho; PR sensível desacelera com intenção; a documentação deixa de correr atrás do produto e passa a nascer junto com ele.

Apêndices — implementação completa dos arquivos principais

Os arquivos abaixo foram modelados para um repositório Python/FastAPI com Docusaurus em `docs-site/`. Ajuste paths, comandos e labels conforme a realidade do projeto. Em produção, siga a recomendação do GitHub e substitua tags amplas de actions por SHAs completos sempre que possível. [R13]

Apêndice A — .github/workflows/pr-ci.yml

Arquivo completo sugerido para implantação inicial. Em ambiente regulado, complemente com pinagem por SHA completo, validação de secrets e scanners adicionais conforme a política interna.

```
name: Pull Request CI

on:
  pull_request:
    branches:
      - main
    types:
      - opened
      - synchronize
      - reopened
      - ready_for_review

concurrency:
  group: pr-ci-${{ github.event.pull_request.number }}
  cancel-in-progress: true

permissions:
  contents: read

jobs:
  quality-gates:
    if: github.event.pull_request.draft == false
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          persist-credentials: false

      - name: Setup Python
        uses: actions/setup-python@v6
        with:
          python-version: '3.12'
          cache: 'poetry'

      - name: Install Poetry
        run: pip install poetry==2.1.3

      - name: Install backend dependencies
        run: poetry install --no-interaction --no-root

      - name: Lint
        run: poetry run ruff check .

      - name: Syntax check
        run: poetry run python -m compileall src tests

      - name: Type check
        run: poetry run mypy src

      - name: Tests
        run: |
          mkdir -p reports
```

```
poetry run pytest -q --maxfail=1 \
--disable-warnings \
--junitxml=reports/pytest.xml
```

```
- name: Upload pytest report
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: pytest-report
    path: reports/pytest.xml
```

docs-gates:

```
if: github.event.pull_request.draft == false
runs-on: ubuntu-latest
```

steps:

```
- name: Checkout full history
  uses: actions/checkout@v6
  with:
    fetch-depth: 0
    persist-credentials: false

- name: Setup Python
  uses: actions/setup-python@v6
  with:
    python-version: '3.12'
    cache: 'poetry'

- name: Setup Node
  uses: actions/setup-node@v6
  with:
    node-version: '22'
    cache: 'pnpm'
    cache-dependency-path: docs-site/pnpm-lock.yaml

- name: Enable Corepack
  run: corepack enable

- name: Install Poetry
  run: pip install poetry==2.1.3

- name: Install backend dependencies
  run: poetry install --no-interaction --no-root

- name: Install docs dependencies
  run: pnpm --dir docs-site install --frozen-lockfile

- name: Validate docs freshness policy
  env:
    BASE_SHA: ${ github.event.pull_request.base.sha }
    HEAD_SHA: ${ github.event.pull_request.head.sha }
  run: poetry run python .github/scripts/docs_guardrails.py

- name: Export OpenAPI
  run: poetry run python scripts/export_openapi.py

- name: Generate API reference
  run: pnpm --dir docs-site docusaurus gen-api-docs all

- name: Build docs site
  run: pnpm --dir docs-site build

- name: Upload docs preview artifact
  uses: actions/upload-artifact@v4
  with:
    name: docs-preview
    path: docs-site/build
```

Apêndice B — .github/workflows/pr-approval.yml

Arquivo completo sugerido para implantação inicial. Em ambiente regulado, complemente com pinagem por SHA completo, validação de secrets e scanners adicionais conforme a política interna.

```
name: Pull Request Approval Bot

on:
  workflow_run:
    workflows:
      - Pull Request CI
    types:
      - completed

permissions:
  contents: read
  pull-requests: write

jobs:
  approve:
    if: >-
      github.event.workflow_run.conclusion == 'success'
    runs-on: ubuntu-latest
    steps:
      - name: Setup Python
        uses: actions/setup-python@v6
        with:
          python-version: '3.12'

      - name: Resolve PR number
        id: pr
        run: |
          python - <<'PY'
          import json, os
          data = json.load(open(os.environ['GITHUB_EVENT_PATH']))
          prs = data.get('workflow_run', {}).get('pull_requests', [])
          pr_number = prs[0]['number'] if prs else ""
          with open(os.environ['GITHUB_OUTPUT'], 'a', encoding='utf-8') as fh:
              fh.write(f'pr_number={pr_number}\n')
          PY

      - name: Stop when workflow_run has no PR
        if: steps.pr.outputs.pr_number == ""
        run: echo 'No pull request associated with this workflow_run.'

      - name: Execute approval policy
        if: steps.pr.outputs.pr_number != ""
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
          GITHUB_REPOSITORY: ${ github.repository }
          PR_NUMBER: ${ steps.pr.outputs.pr_number }
        run: python .github/scripts/approval_policy.py
```

Apêndice C — .github/scripts/approval_policy.py

Arquivo completo sugerido para implantação inicial. Em ambiente regulado, complemente com pinagem por SHA completo, validação de secrets e scanners adicionais conforme a política interna.

```
from __future__ import annotations
```

```

import json
import os
import re
import sys
import urllib.request
from urllib.error import HTTPError

API = 'https://api.github.com'
TOKEN = os.environ['GITHUB_TOKEN']
REPO = os.environ['GITHUB_REPOSITORY']
PR_NUMBER = os.environ['PR_NUMBER']
BOT_LOGIN = 'github-actions[bot]'

BLOCKING_LABELS = {
    'security',
    'breaking-change',
    'db-migration',
    'infra-change',
    'needs-human-review',
}

PROTECTED_PATTERNS = [
    r'^\.github/',
    r'^infra/',
    r'^terraform/',
    r'^helm/',
    r'^migrations?/',
    r'^alembic/',
    r'^docs-site/docusaurus\.config\.(js|ts)$',
    r'^docs-site/sidebars\.(js|ts)$',
    r'^Dockerfile',
    r'^pyproject\.toml$',
    r'^poetry\.lock$',
    r'^package\.json$',
    r'^pnpm-lock\.yaml$',
]

MAX_CHANGED_FILES = 30
MAX_TOTAL_DELTA = 800

def api_request(method: str, path: str, payload: dict | None = None):
    req = urllib.request.Request(
        f'{API}{path}',
        method=method,
        headers={
            'Authorization': f'Bearer {TOKEN}',
            'Accept': 'application/vnd.github+json',
            'X-GitHub-API-Version': '2022-11-28',
        },
        data=(json.dumps(payload).encode('utf-8') if payload else None),
    )
    try:
        with urllib.request.urlopen(req) as resp:
            return json.loads(resp.read().decode('utf-8'))
    except HTTPError as exc:
        body = exc.read().decode('utf-8', errors='ignore')
        raise RuntimeError(f'GitHub API error {exc.code}: {body}')

def paginate(path: str):
    page = 1
    while True:
        chunk = api_request('GET', f'{path}?per_page=100&page={page}')
        if not chunk:
            break

```

```

    yield from chunk
    if len(chunk) < 100:
        break
    page += 1

def fail_closed(reason: str):
    print(reason)
    sys.exit(0)

def main():
    pr = api_request('GET', f'/repos/{REPO}/pulls/{PR_NUMBER}')

    if pr['state'] != 'open':
        fail_closed('PR is not open; no approval will be registered.')
    if pr.get('draft'):
        fail_closed('PR is still a draft; no approval will be registered.')
    if pr['head']['repo']['fork']:
        fail_closed('PR comes from a fork; policy forbids auto-approval.')

    labels = {item['name'] for item in pr.get('labels', [])}
    matched_labels = BLOCKING_LABELS.intersection(labels)
    if matched_labels:
        fail_closed(f'Blocking labels found: {sorted(matched_labels)}')

    files = list(paginate(f'/repos/{REPO}/pulls/{PR_NUMBER}/files'))
    if len(files) > MAX_CHANGED_FILES:
        fail_closed(f'Too many changed files ({len(files)}).')

    total_delta = sum(item.get('additions', 0) + item.get('deletions', 0) for item in files)
    if total_delta > MAX_TOTAL_DELTA:
        fail_closed(f'Total diff too large ({total_delta} lines).')

    filenames = [item['filename'] for item in files]
    for name in filenames:
        if any(re.search(pattern, name) for pattern in PROTECTED_PATTERNS):
            fail_closed(f'Protected path touched: {name}')

    reviews = list(paginate(f'/repos/{REPO}/pulls/{PR_NUMBER}/reviews'))
    for review in reviews:
        if review.get('state') == 'CHANGES_REQUESTED':
            fail_closed('There is at least one CHANGES_REQUESTED review.')
        if review.get('state') == 'APPROVED' and review.get('user', {}).get('login') == BOT_LOGIN:
            fail_closed('Approval already registered by the bot.')

    body = (
        'Aprovacao automatica registrada. '\
        'Todos os quality gates do workflow Pull Request CI passaram, '\
        'o PR nao esta em draft, nao veio de fork, nao possui labels bloqueantes '\
        'e nao tocou caminhos protegidos pela politica de auto-approval.'
    )
    api_request(
        'POST',
        f'/repos/{REPO}/pulls/{PR_NUMBER}/reviews',
        {'event': 'APPROVE', 'body': body},
    )
    print('Approval review submitted successfully.')

if __name__ == '__main__':
    main()

```

Apêndice D — .github/scripts/docs_guardrails.py

Arquivo completo sugerido para implantação inicial. Em ambiente regulado, complemente com pinagem por SHA completo, validação de secrets e scanners adicionais conforme a política interna.

```
from __future__ import annotations

import os
import subprocess
import sys
from pathlib import Path

BASE_SHA = os.environ['BASE_SHA']
HEAD_SHA = os.environ['HEAD_SHA']

PUBLIC_CHANGE_PREFIXES = [
    'src/',
    'app/',
    'api/',
    'openapi/',
]

DOC_TOUCH_PREFIXES = [
    'docs-site/docs/',
    'docs-site/openapi/',
    'docs/',
    'README.md',
    'CHANGELOG.md',
]

IGNORED_CODE_PREFIXES = [
    'tests/',
    'docs-site/',
    '.github/',
]

def changed_files() -> list[str]:
    cmd = ['git', 'diff', '--name-only', f'{BASE_SHA}...{HEAD_SHA}']
    return [line.strip() for line in subprocess.check_output(cmd, text=True).splitlines() if line.strip()]

def main():
    files = changed_files()
    public_changes = [
        f for f in files
        if any(f.startswith(prefix) for prefix in PUBLIC_CHANGE_PREFIXES)
        and not any(f.startswith(prefix) for prefix in IGNORED_CODE_PREFIXES)
    ]
    doc_changes = [
        f for f in files
        if any(f == prefix or f.startswith(prefix) for prefix in DOC_TOUCH_PREFIXES)
    ]

    if public_changes and not doc_changes:
        print('Public-facing code changed but no documentation path was touched.')
        print('Changed public files:')
        for item in public_changes:
            print(f' - {item}')
        sys.exit(1)

    print('Documentation freshness policy passed.')

if __name__ == '__main__':
    main()
```

Apêndice E — .github/CODEOWNERS

Arquivo completo sugerido para implantação inicial. Em ambiente regulado, complemente com pinagem por SHA completo, validação de secrets e scanners adicionais conforme a política interna.

```
# Default owners
*                @org/backend-platform

# CI/CD and repository policy
/github/         @org/platform-engineering @org/security
/github/workflows/ @org/platform-engineering @org/security

# Infrastructure
/infra/         @org/platform-engineering
/terraform/     @org/platform-engineering
/helm/          @org/platform-engineering
Dockerfile*     @org/platform-engineering

# Database and migrations
/migrations/    @org/data-platform @org/backend-platform
/alembic/       @org/data-platform @org/backend-platform

# Application API / public contracts
/src/           @org/backend-platform
/openapi/       @org/backend-platform @org/tech-writers

# Documentation portal
/docs-site/docusaurus.config.ts @org/tech-writers @org/platform-engineering
/docs-site/sidebars.ts          @org/tech-writers
/docs-site/docs/                @org/tech-writers
```

Apêndice F — scripts/export_openapi.py

Arquivo completo sugerido para implantação inicial. Em ambiente regulado, complemente com pinagem por SHA completo, validação de secrets e scanners adicionais conforme a política interna.

```
from __future__ import annotations

import json
from pathlib import Path

from app.main import app

def main() -> None:
    out = Path('docs-site/openapi/openapi.json')
    out.parent.mkdir(parents=True, exist_ok=True)
    schema = app.openapi()
    out.write_text(json.dumps(schema, ensure_ascii=False, indent=2), encoding='utf-8')
    print(f'OpenAPI exported to {out}')

if __name__ == '__main__':
    main()
```

Apêndice G — docs-site/package.json

Arquivo completo sugerido para implantação inicial. Em ambiente regulado, complemente com pinagem por SHA completo, validação de secrets e scanners adicionais conforme a política interna.

```
{
  "name": "company-docs",
  "private": true,
  "packageManager": "pnpm@10.6.3",
  "scripts": {
    "start": "docusaurus start",
    "build": "docusaurus build",
    "serve": "docusaurus serve",
    "clear": "docusaurus clear",
    "typecheck": "tsc --noEmit",
    "gen:api": "docusaurus gen-api-docs all",
    "version:docs": "docusaurus docs:version"
  },
  "dependencies": {
    "@docusaurus/core": "^3.9.2",
    "@docusaurus/preset-classic": "^3.9.2",
    "docusaurus-plugin-openapi-docs": "^4.7.1",
    "docusaurus-theme-openapi-docs": "^4.7.1",
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  },
  "devDependencies": {
    "@docusaurus/module-type-aliases": "^3.9.2",
    "@docusaurus/tsconfig": "^3.9.2",
    "@docusaurus/types": "^3.9.2",
    "typescript": "^5.8.0"
  }
}
```

Apêndice H — docs-site/docusaurus.config.ts

Arquivo completo sugerido para implantação inicial. Em ambiente regulado, complemente com pinagem por SHA completo, validação de secrets e scanners adicionais conforme a política interna.

```
import type {Config} from '@docusaurus/types';
import type * as Preset from '@docusaurus/preset-classic';
import type * as OpenApiPlugin from 'docusaurus-plugin-openapi-docs';

const config: Config = {
  title: 'Engineering Docs',
  tagline: 'Documentacao viva de produto, plataforma e operacao',
  favicon: 'img/favicon.ico',
  url: 'https://example.github.io',
  baseUrl: '/',
  organizationName: 'example',
  projectName: 'engineering-docs',
  onBrokenLinks: 'throw',
  onBrokenMarkdownLinks: 'throw',
  i18n: {
    defaultLocale: 'pt-BR',
    locales: ['pt-BR'],
  },
  presets: [
    [
      'classic',
      {

```

```

docs: {
  routeBasePath: '/',
  sidebarPath: './sidebars.ts',
  docItemComponent: '@theme/ApiItem',
  showLastUpdateAuthor: true,
  showLastUpdateTime: true,
  editUrl: 'https://github.com/example/repo/tree/main/docs-site/',
  lastVersion: 'current',
  versions: {
    current: {
      label: 'Next',
      path: 'next',
      banner: 'unreleased',
    },
  },
  blog: false,
  theme: {
    customCss: './src/css/custom.css',
  },
} satisfies Preset.Options,
],
plugins: [
  [
    'docusaurus-plugin-openapi-docs',
    {
      id: 'api',
      docsPluginId: 'classic',
      config: {
        core: {
          specPath: 'openapi/openapi.json',
          outputDir: 'docs/api/core',
          sidebarOptions: {
            groupPathsBy: 'tag',
            sidebarCollapsible: true,
            sidebarCollapsed: true,
          },
        },
      } satisfies OpenApiPlugin.Options,
    },
  ],
],
themes: ['docusaurus-theme-openapi-docs'],
themeConfig: {
  image: 'img/social-card.png',
  navbar: {
    title: 'Engineering Docs',
    items: [
      {to: '/', label: 'Home', position: 'left'},
      {to: '/architecture/overview', label: 'Architecture', position: 'left'},
      {to: '/standards/coding-standards', label: 'Standards', position: 'left'},
      {to: '/runbooks/deploy', label: 'Runbooks', position: 'left'},
      {type: 'docsVersionDropdown', position: 'right'},
      {
        href: 'https://github.com/example/repo',
        label: 'GitHub',
        position: 'right',
      },
    ],
  },
},
footer: {
  style: 'dark',
  links: [
    {
      title: 'Portal',
      items: [

```

```

    {label: 'Architecture', to: '/architecture/overview'},
    {label: 'Standards', to: '/standards/coding-standards'},
    {label: 'Runbooks', to: '/runbooks/deploy'},
    {label: 'API', to: '/api/core'},
  ],
},
],
copyright: `Copyright © ${new Date().getFullYear()} Example Corp`,
},
},
};

export default config;

```

Apêndice I — docs-site/sidebars.ts

Arquivo completo sugerido para implantação inicial. Em ambiente regulado, complemento com pinagem por SHA completo, validação de secrets e scanners adicionais conforme a política interna.

```

import type {SidebarsConfig} from '@docusaurus/plugin-content-docs';

const sidebars: SidebarsConfig = {
  portalSidebar: [
    'index',
    {
      type: 'category',
      label: 'Architecture',
      link: {type: 'generated-index'},
      items: [{type: 'autogenerated', dirName: 'architecture'}],
    },
    {
      type: 'category',
      label: 'Standards',
      link: {type: 'generated-index'},
      items: [{type: 'autogenerated', dirName: 'standards'}],
    },
    {
      type: 'category',
      label: 'Runbooks',
      link: {type: 'generated-index'},
      items: [{type: 'autogenerated', dirName: 'runbooks'}],
    },
    {
      type: 'category',
      label: 'ADR',
      link: {type: 'generated-index'},
      items: [{type: 'autogenerated', dirName: 'adr'}],
    },
    {
      type: 'category',
      label: 'API Reference',
      link: {type: 'generated-index'},
      items: [{type: 'autogenerated', dirName: 'api'}],
    },
  ],
};

export default sidebars;

```

Apêndice J — .github/workflows/docs-deploy.yml

Arquivo completo sugerido para implantação inicial. Em ambiente regulado, complemente com pinagem por SHA completo, validação de secrets e scanners adicionais conforme a política interna.

```
name: Docs Deploy

on:
  push:
    branches:
      - main

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: docs-deploy
  cancel-in-progress: true

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout full history
        uses: actions/checkout@v6
        with:
          fetch-depth: 0

      - name: Setup Python
        uses: actions/setup-python@v6
        with:
          python-version: '3.12'
          cache: 'poetry'

      - name: Setup Node
        uses: actions/setup-node@v6
        with:
          node-version: '22'
          cache: 'pnpm'
          cache-dependency-path: docs-site/pnpm-lock.yaml

      - name: Configure GitHub Pages
        uses: actions/configure-pages@v5

      - name: Enable Corepack
        run: corepack enable

      - name: Install Poetry
        run: pip install poetry==2.1.3

      - name: Install backend dependencies
        run: poetry install --no-interaction --no-root

      - name: Install docs dependencies
        run: pnpm --dir docs-site install --frozen-lockfile

      - name: Export OpenAPI
        run: poetry run python scripts/export_openapi.py

      - name: Generate API reference
        run: pnpm --dir docs-site docusaurus gen-api-docs all

      - name: Build portal
        run: pnpm --dir docs-site build
```

```

- name: Upload Pages artifact
  uses: actions/upload-pages-artifact@v4
  with:
    path: docs-site/build

deploy:
  environment:
    name: github-pages
    url: ${ steps.deployment.outputs.page_url }
  runs-on: ubuntu-latest
  needs: build
  steps:
    - name: Deploy to GitHub Pages
      id: deployment
      uses: actions/deploy-pages@v4

```

Apêndice K — .github/workflows/docs-version-pr.yml

Arquivo completo sugerido para implantação inicial. Em ambiente regulado, complemente com pinagem por SHA completo, validação de secrets e scanners adicionais conforme a política interna.

```

name: Docs Version Freeze PR

on:
  push:
    tags:
      - 'v*.*.*'

permissions:
  contents: write
  pull-requests: write

jobs:
  freeze-docs-version:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout default branch
        uses: actions/checkout@v6
        with:
          ref: main
          fetch-depth: 0

      - name: Setup Python
        uses: actions/setup-python@v6
        with:
          python-version: '3.12'
          cache: 'poetry'

      - name: Setup Node
        uses: actions/setup-node@v6
        with:
          node-version: '22'
          cache: 'pnpm'
          cache-dependency-path: docs-site/pnpm-lock.yaml

      - name: Enable Corepack
        run: corepack enable

      - name: Install Poetry
        run: pip install poetry==2.1.3

      - name: Install backend dependencies

```

```

run: poetry install --no-interaction --no-root

- name: Install docs dependencies
run: pnpm --dir docs-site install --frozen-lockfile

- name: Export OpenAPI
run: poetry run python scripts/export_openapi.py

- name: Generate latest API docs
run: pnpm --dir docs-site docusaurus gen-api-docs all

- name: Freeze docs version
env:
  VERSION: ${GITHUB_REF_NAME}
run: |
  VERSION_CLEAN="${VERSION#v}"
  pnpm --dir docs-site docusaurus docs:version "$VERSION_CLEAN"
  echo "VERSION_CLEAN=$VERSION_CLEAN" >> "$GITHUB_ENV"

- name: Commit version freeze to automation branch
run: |
  BRANCH="automation/docs-version-${VERSION_CLEAN}"
  git checkout -b "$BRANCH"
  git config user.name "github-actions[bot]"
  git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
  git add docs-site/versioned_docs docs-site/versioned_sidebars docs-site/versions.json
  git commit -m "docs: freeze version ${VERSION_CLEAN}" || exit 0
  git push origin "$BRANCH"
  echo "BRANCH=$BRANCH" >> "$GITHUB_ENV"

- name: Open pull request
env:
  GH_TOKEN: ${GITHUB_TOKEN}
run: |
  python - <<'PY'
  import json, os, urllib.request
  repo = os.environ['GITHUB_REPOSITORY']
  token = os.environ['GH_TOKEN']
  branch = os.environ['BRANCH']
  version = os.environ['VERSION_CLEAN']
  payload = {
    'title': f'docs: freeze version {version}',
    'head': branch,
    'base': 'main',
    'body': (
      'PR automatizado para congelar a fotografia documental da release '\
      f'{version}. Revisar e mesclar para publicar a nova versao do portal.'
    ),
  }
  req = urllib.request.Request(
    f'https://api.github.com/repos/{repo}/pulls',
    method='POST',
    data=json.dumps(payload).encode('utf-8'),
    headers={
      'Authorization': f'Bearer {token}',
      'Accept': 'application/vnd.github+json',
      'X-GitHub-Api-Version': '2022-11-28',
    },
  )
  try:
    with urllib.request.urlopen(req) as resp:
      print(resp.read().decode('utf-8'))
  except Exception as exc:
    print(f'Could not create PR automatically: {exc}')
    raise
PY

```


Referências

Fontes consultadas em 01/04/2026. Priorizei documentação oficial do GitHub e do Docusaurus; o plugin OpenAPI foi incluído como referência comunitária por ser um componente prático para a estratégia de API docs descrita neste blueprint.

ID	Fonte	Local	Tipo
R1	Docusaurus — Introdução / visão geral do produto	docusaurus.io/docs	fonte oficial
R2	Docusaurus — Installation	docusaurus.io/docs/installation	fonte oficial
R3	Docusaurus — TypeScript Support	docusaurus.io/docs/typescript-support	fonte oficial
R4	Docusaurus — Versioning	docusaurus.io/docs/versioning	fonte oficial
R5	Docusaurus docs plugin — showLastUpdateTime e fetch-depth: 0	docusaurus.io/docs/api/plugins/@docusaurus/plugin-content-docs	fonte oficial
R6	Docusaurus — Deployment / Search	docusaurus.io/docs/deployment + docusaurus.io/docs/search	fonte oficial
R7	GitHub Docs — About protected branches	docs.github.com/.../about-protected-branches	fonte oficial
R8	GitHub Docs — About rulesets	docs.github.com/.../about-rulesets	fonte oficial
R9	GitHub Docs — About status checks	docs.github.com/articles/about-status-checks	fonte oficial
R10	GitHub Docs — About pull request reviews	docs.github.com/articles/about-pull-request-reviews	fonte oficial
R11	GitHub Docs — About code owners	docs.github.com/articles/about-code-owners	fonte oficial
R12	GitHub Docs — Events that trigger workflows	docs.github.com/actions/using-workflows/events-that-trigger-workflows	fonte oficial
R13	GitHub Docs — Secure use reference	docs.github.com/en/actions/reference/security/secure-use	fonte oficial
R14	GitHub Docs — Use GITHUB_TOKEN for authentication in workflows	docs.github.com/actions/reference/authentication-in-a-workflow	fonte oficial
R15	GitHub Docs — Managing GitHub Actions settings for a repository	docs.github.com/.../managing-github-actions-settings-for-a-repository	fonte oficial
R16	GitHub Docs — Using custom workflows with GitHub Pages	docs.github.com/.../using-custom-workflows-with-github-pages	fonte oficial
R17	actions/checkout	github.com/actions/checkout	ação oficial
R18	actions/setup-python	github.com/actions/setup-python	ação oficial
R19	actions/setup-node	github.com/actions/setup-node	ação oficial

R20	PaloAltoNetworks/ docusaurus-openapi-docs	github.com/ PaloAltoNetworks/ docusaurus-openapi-docs	componente comunitário
-----	--	---	------------------------